

2차 과제에서 만든 작업자 생산성 평가 시스템을 3차 과제 기준에 맞게 메뉴와 함수 중심 구조로 확장하면서 내용을 구체화하고 싶어

좋습니다.

3차 과제에서는 2차 과제에서 만든 **작업자 생산성 평가 시스템**을 단순 실행형 코드에서 **메뉴 기반 관리 프로그램**으로 확장하면 좋습니다.

2차 과제에서는 여러 작업자의 이름, 작업량, 작업 시간을 리스트에 저장하고, 생산성을 계산한 뒤 등급과 특별 평가를 출력하는 구조였습니다.

3차 과제에서는 이 기능들을 하나의 긴 코드로 작성하기보다, 역할별로 함수로 나누는 것이 핵심입니다.

예를 들어 전체 구조는 다음과 같이 잡을 수 있습니다.

1. 작업자 정보 입력

- 작업자 이름, 작업량, 작업 시간을 입력받음
- 생산성을 계산하여 리스트에 저장

2. 전체 작업자 조회

- 저장된 작업자들의 이름, 작업량, 작업 시간, 생산성, 등급 출력

3. 생산성 분석

- 평균 생산성 계산
- 최고 생산성 작업자 찾기
- 생산성 정렬 결과 출력



+ 무엇이든 물어보세요

Instant ▾



3. 생산성 분석

- 평균 생산성 계산
- 최고 생산성 작업자 찾기
- 생산성 정렬 결과 출력

4. 프로그램 종료

- 사용자가 종료를 선택할 때만 `break` 로 종료

함수는 다음처럼 나누면 적절합니다.

<> Python



▶ 실행됨

```
calculate_productivity()
add_worker()
show_workers()
analyze_productivity()
```

이렇게 구성하면 2차 과제에서 사용한 리스트, 조건문, 반복문은 유지하면서도 3차 과제에서 요구하는 `while True`, 메뉴 구조, 함수 분리, 매개변수, `return`, 전역변수 활용까지 자연스럽게 반영할 수 있습니다.

📄 📌 🔍 🗑️ ↺ ...

작업자 정보 입력, ↓ 산성 계산, 결과 조회 기능을 함수로 나누고, 전역 변수와 지역변수를 구분해서 사용하는 구조를 설계해줘

+ 무엇이든 물어보세요

Instant ▾



작업자 정보 입력, 생산성 계산, 결과 조회 기능을 담당하도록 하고, 전역 변수와 지역변수를 구분해서 사용하는 구조를 설계해줘

좋습니다.

3차 과제에서는 기능을 함수로 분리하고, 데이터를 저장하는 부분은 전역변수, 함수 안에서 잠깐 사용하는 값은 지역변수로 구분하면 됩니다.

전체 구조는 다음과 같이 설계하면 좋습니다.

1. 전역변수 설계

작업자 정보는 프로그램 전체에서 계속 사용해야 하므로 전역 리스트로 관리합니다.

Python

```
worker_names = []
work_amounts = []
work_times = []
productivities = []
grades = []
```



▶ 실행됨

이 리스트들은 작업자 입력, 전체 조회, 생산성 분석 ↓에서 모두 사용되므로 전역변수로 두는 것이 적절합니다.

+ 무엇이든 물어보세요

Instant ▾



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.

2. 함수 구조 설계

1) 생산성 계산 함수

Python

```
def calculate_productivity(work_amount, work_time):
    productivity = work_amount / work_time
    return productivity
```



▶ 실행됨

이 함수는 작업량과 작업 시간을 매개변수로 받아 생산성을 계산하고, 결과를 `return` 으로 반환합니다. 여기서 `productivity` 는 함수 안에서만 쓰이는 지역변수입니다.

2) 작업자 정보 입력 함수

Python

```
def add_worker():
    global worker_names, work_amounts, work_times, productivities, grades
```



▶ 실행됨

이 함수는 작업자 정보를 입력받고, 전역 리스트에 데이터를 추가하는 역할을 합니다.

전역 리스트에 값을 저장해야 하므로 `global` 을 사용 ↓합니다.

함수 내부에서 입력받는 `name`, `work_amount`, `work_time`, `productivity`, `grade` 는 함수 안에서 만들어지는 지역변수입니다

함수 내부에서 입력받는 `name`, `work_amount`, `work_time`, `productivity`, `grade` 는 함수 안에서 만들어지는 지역변수입니다.
하지만 최종 결과는 전역 리스트에 저장되기 때문에 다른 함수에서도 사용할 수 있습니다.

3) 결과 조회 함수

<> Python



▶ 실행됨

```
def show_workers():
```

이 함수는 전역 리스트에 저장된 작업자 정보를 출력하는 기능을 담당합니다.
저장된 데이터만 읽기 때문에 `global` 을 꼭 사용하지 않아도 됩니다.

4) 생산성 분석 함수

<> Python



▶ 실행됨

```
def analyze_productivity():
```



이 함수는 저장된 생산성 리스트를 활용하여 평균 생산성, 최고 생산성, 최고 작업자를 계산합니다.
여기서 `total`, `average`, `max_productivity`, `top_index` 는 함수 내부에서만 사용되는 지역변수입니다.

+ 무엇이든 물어보세요

Instant ▾



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.

이 함수는 저장된 생산성 리스트를 활용하여 평균 생산성, 최고 생산성, 최고 작업자를 계산합니다.
여기서 `total`, `average`, `max_productivity`, `top_index` 는 함수 내부에서만 사용되는 지역변수입니다.

3. 전체 흐름

전역 리스트 생성



↓

메인 메뉴 `while True` 실행

↓

1번 선택 → `add_worker()`

2번 선택 → `show_workers()`

3번 선택 → `analyze_productivity()`

4번 선택 → `break`로 종료

이렇게 설계하면 전역변수는 프로그램 전체 데이터 저장용, 지역변수는 함수 내부 계산용으로 역할이 명확하게 나뉩니다.

또한 3차 과제에서 요구하는 함수 분리, 매개변수, 반환값, `global` 사용 조건을 모두 자연스럽게 충족할 수 있습니다.

출처

+ 무엇이든 물어보세요

Instant ▾



ChatGPT는 실수를 할 수 있습니다. 중요한 정보는 재차 확인하세요.